

Project Report: Week 6

Summary & Reinforcement Learning Revision

1. Foundations of Reinforcement Learning (RL)

Reinforcement Learning introduces a trial-and-error paradigm where an **Agent** interacts with an **Environment**. The agent observes an environmental **State**, executes an **Action**, receives a scalar **Reward**, and transitions to a **Next State**. The ultimate goal is to optimize a behavioral **Policy** that maximizes cumulative long-term rewards.

While Week 4 used deep learning to capture sequence patterns, Week 6 leverages Deep Learning to approximate the complex value and policy spaces required for dynamic decision-making.

2. Deep Q-Networks (DQN)

DQN adapts discrete value-based model-free RL (Q-Learning) to handle massive state spaces by replacing traditional tabular lookup grids with a Deep Neural Network parameterizer.

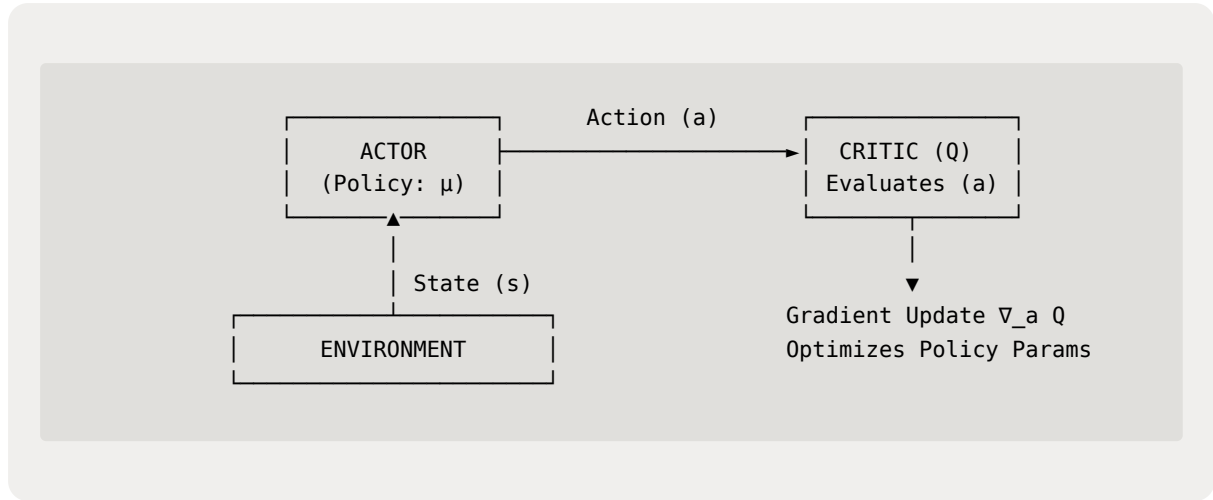
Core Architecture Components:

- **Neural Network Quantizer:** Maps a high-dimensional state vector down to a output vector of Q-values representing every possible discrete action.
- **Experience Replay Buffer:** Past transition tuples are held in a memory cache. Training updates draw random mini-batches from this pool, which breaks the temporal correlation of sequential tracking and improves stability.
- **Target Network Convergence:** To prevent pursuit loops during updates, a detached target network evaluates target indices. Its weights are held constant and updated periodically to match the main network.
- **Loss Function Formulation:** Minimizes the mean squared bellman error:

$$L(\theta) = \mathbb{E} \left[\left(r + \gamma \max_{a'} Q(s', a'; \theta^-) - Q(s, a; \theta) \right)^2 \right]$$

3. Deep Deterministic Policy Gradient (DDPG)

While DQN handles discrete actions, DDPG is an off-policy, model-free **Actor-Critic** architecture engineered to resolve tasks in continuous action spaces.



Core Structural Tenets:

- **Deterministic Policy:** Instead of computing broad probability distributions, the **Actor network** maps an environmental state directly to an exact continuous action scalar.
- **Critic Network:** The **Critic** $Q(s, a|\theta^Q)$ evaluates the actor's performance by estimating the expected Q-value of the state-action pair.
- **Action Selection Rule:** Explores continuous landscapes by adding a directional random noise process to the deterministic actor output:

$$a_t = \mu(s_t|\theta^\mu) + \mathcal{N}_t$$

- **Soft Target Network Updates:** Instead of copying weights abruptly, target parameters are slowly updated using a tracking factor to smooth training transitions:

$$\theta^- \leftarrow \tau\theta + (1 - \tau)\theta^-$$

4. Reinforcement Learning Pseudocode Revision Framework

This layout illustrates how a generic deep reinforcement learning environment loop operates, capturing the experience gathering and updating mechanics.

```
import numpy as np
import random
from collections import deque

# 1. Experience Replay Initialization
replay_buffer = deque(maxlen=2000)
epsilon = 1.0          # Exploration rate factor
epsilon_decay = 0.995  # Decay multiplier per episode
gamma = 0.95           # Discount factor for future rewards

# 2. Simulation Environment Action Interception
def interact_and_store(state, action, reward, next_state, done):
    # Store transition sample vector inside cache
    replay_buffer.append((state, action, reward, next_state, done))

# 3. Action Selection via Epsilon-Greedy Logic
def select_action(state, model, action_space_size):
    global epsilon
    if np.random.rand() <= epsilon:
        return random.randrange(action_space_size) # Explore: Choose random action

    q_values = model.predict(state, verbose=0)
    return np.argmax(q_values[0])                 # Exploit: Choose best action
```

Code Term Breakdown:

- `deque(maxlen=2000)` : Implements the experience replay buffer container. Once the memory footprint reaches 2,000 transition records, it drops the oldest row to accommodate fresh updates.
- `epsilon = 1.0` : Initializes the exploration parameter for the epsilon-greedy strategy. It ensures the agent starts by fully exploring its environment before gradually shifting toward exploiting its learned policy as `epsilon` decays.
- `np.argmax(q_values[0])` : Selects the action that maximizes the network's predicted future cumulative reward for the current state.

5. Assigned Benchmark Challenge: Classic Inverted Pendulum

- **Problem Objective:** Balance a pole upright on a movable cart base against gravity.

- **Action Constraints:** The agent can apply force to swing the base either left or right.
- **Application Context:** This serves as a primary verification step[cite: 1]. Validating the agent's performance on this classic control problem ensures the replay buffer logic, network synchronization, and reward tracking are working correctly before deploying the model in financial environments.